IST604 — Web Services and Distributed Computing

Comprehensive Practice Examination (140 Marks)

Coverage: Distributed computing, SOA, SOAP, REST, WSDL, UDDI, JAX-WS, MOM, CORBA/DCOM/RMI, HTTP
Based on: Course lectures, summaries, CA (June 2025), First Semester Exam (27 June 2025)

Section	Format	Marks
A	Multiple Choice (35 questions)	35
B	Structured / Short Answer	35
C	Essay	70
Total		**140**

SECTION A — MULTIPLE CHOICE (35 marks)

*One mark each. Choose the **best** answer.*

---

1. Distributed computing is best described as:

- (a) Running one program on one very fast computer
- (b) Multiple computers working together so networked components coordinate to solve a common problem
- (c) Storing all data in a single central database
- (d) Using only peer-to-peer file sharing

Answer: (b)

---

2. Which is **not** a typical benefit of distributed computing emphasized in the course?

- (a) Performance through parallel execution
- (b) Resilience when one node fails
- (c) Guaranteed elimination of all network latency
- (d) Reuse of shared network-accessible components

Answer: (c)

---

3. CORBA is characterized as:

- (a) Microsoft-only, Windows-to-Windows
- (b) Java-only remote method calls
- (c) Language-neutral, OMG standard object request broker
- (d) A REST architectural style

Answer: (c)

---

4. Java RMI differs from generic `java.net` networking because RMI:

- (a) Only works with XML messages
- (b) Provides transparent distributed **Java object** invocation using JRMP between JVMs
- (c) Is the same as HTTP
- (d) Requires UDDI

Answer: (b)

---

5. Message-Oriented Middleware (MOM) is primarily:

- (a) Synchronous and tightly coupled like RPC
- (b) Asynchronous, loosely coupled communication via message queues
- (c) A replacement for HTML
- (d) Only used by DCOM

Answer: (b)

---

6. A **web service** must:

- (a) Always use SOAP
- (b) Operate over a network using standardized protocols
- (c) Be written only in Java
- (d) Replace all local APIs

Answer: (b)

---

7. All web services are APIs, but not all APIs are web services because:

- (a) APIs never use HTTP
- (b) Some APIs are local (e.g. OS libraries) and need not use a network
- (c) Web services cannot return data
- (d) APIs are always proprietary

Answer: (b)

---

8. In SOA, the **Service Broker/Registry** role is typically associated with:

- (a) SOAP
- (b) WSDL only
- (c) UDDI
- (d) JRMP

Answer: (c)

---

9. SOA is to web services as:

- (a) Implementation is to blueprint (reversed)
- (b) Architectural blueprint is to technical implementation
- (c) They are unrelated
- (d) REST is to SOAP

Answer: (b)

---

10. RPC-based communication in web services is:

- (a) Loosely coupled and asynchronous
- (b) Tightly coupled and typically synchronous
- (c) Always document-driven
- (d) Never used with CORBA or RMI

Answer: (b)

---

11. Which SOAP element is **required**?

- (a) Header
- (b) Fault
- (c) Body
- (d) Attachment

Answer: (c) *(Envelope is also required as root; Body is required content container.)*

---

12. SOAP **Fault** elements appear:

- (a) In the Header only
- (b) Inside the Body when errors occur
- (c) Outside the Envelope
- (d) Only in REST responses

Answer: (b)

---

13. WSDL is primarily:

- (a) A transport protocol
- (b) An XML contract describing service operations, types, bindings, and endpoints
- (c) A Java annotation
- (d) A JSON schema only

Answer: (b)

---

14. In WSDL, `<portType>` (WSDL 1.1) groups:

- (a) Physical URL addresses only

- (b) Abstract operations and their input/output messages
- (c) Only XSD types
- (d) HTTP status codes

Answer: (b)

---

15. UDDI stands for:

- (a) Universal Data Distribution Interface
- (b) Universal Description, Discovery, and Integration
- (c) Unified Document Definition Index
- (d) User-Driven Data Interchange

Answer: (b)

---

16. REST is best classified as:

- (a) A strict W3C messaging protocol like SOAP
- (b) An architectural style using HTTP methods and resource-oriented URLs
- (c) A Microsoft proprietary standard
- (d) A DTD replacement

Answer: (b)

---

17. Which HTTP verb maps to **Delete** in CRUD?

- (a) GET
- (b) POST
- (c) PUT
- (d) DELETE

Answer: (d)

---

18. Compared to SOAP, REST typically:

- (a) Uses only XML and is slower
- (b) Uses JSON more often, supports HTTP caching, and is lighter
- (c) Requires WSDL for every call
- (d) Cannot work over HTTPS

Answer: (b)

---

19. In JAX-WS, the **SEI** is:

- (a) The class that publishes the endpoint
- (b) The Service Endpoint Interface declaring web operations
- (c) The SOAP Envelope
- (d) The UDDI registry

Answer: (b)

---

20. `@SOAPBinding(style = SOAPBinding.Style.RPC)` indicates:

- (a) Document-style SOAP (default)
- (b) RPC-style SOAP binding
- (c) REST binding
- (d) No SOAP is used

Answer: (b)

---

21. If `@SOAPBinding` is **omitted** in JAX-WS, the default style is:

- (a) RPC
- (b) Document
- (c) REST
- (d) None

Answer: (b)

---

22. `Endpoint.publish(url, implementation)` — the second argument is:

- (a) The WSDL file path
- (b) An instance of the **SIB** (service implementation)
- (c) The client proxy
- (d) UDDI entry

Answer: (b)

---

23. `wsimport` is used by:

- (a) Service providers to generate WSDL from code
- (b) Service consumers to generate client stubs from WSDL (top-down)
- (c) Only REST clients
- (d) DNS configuration

Answer: (b)

---

24. `wsgen` is primarily used for:

- (a) Bottom-up generation of WSDL/artifacts from `@WebService` classes (provider)
- (b) Importing REST OpenAPI specs
- (c) Compiling TypeScript
- (d) UDDI registration only

Answer: (a)

---

25. In a JAX-WS client, `QName(namespaceURI, localName)` identifies:

- (a) The TCP port number only
- (b) The WSDL **service** namespace and service name for `Service.create()`
- (c) The SOAP Fault code
- (d) The HTML title

Answer: (b)

---

26. Three roles in SOAP message transmission include:

- (a) Sender, Intermediary, Ultimate Receiver
- (b) GET, POST, PUT
- (c) Provider, Consumer, Browser only
- (d) CORBA, DCOM, RMI

Answer: (a)

---

27. DCOM is:

- (a) Language-neutral like CORBA
- (b) Microsoft Windows-oriented distributed component technology
- (c) The same as JSON
- (d) A WSDL element

Answer: (b)

---

28. For URL `http://www.example.com:8080/api?id=1#section`, the **fragment** is:

- (a) `http`
- (b) `www.example.com`
- (c) `?id=1`
- (d) `#section`

Answer: (d)

---

29. In HTTP, the `?` in `GET /search?q=test` starts the:

- (a) Fragment
- (b) Query string
- (c) Path
- (d) Scheme

Answer: (b)

---

30. HTTP status code **201 Created** in a REST API typically means:

- (a) Resource not found

- (b) A new resource was successfully created
- (c) Server internal error
- (d) Unauthorized

Answer: (b)

---

31. #PCDATA in a DTD means:

- (a) Data that will not be parsed
- (b) Parsed character data — parser processes entities and markup rules
- (c) Binary attachment only
- (d) REST payload

Answer: (b)

---

32. A **well-formed** XML document must have:

- (a) A DTD or XSD only
- (b) Exactly one root element and correct syntax (nesting, tags, quoted attributes)
- (c) No attributes
- (d) Only JSON content

Answer: (b)

---

33. Valid XML requires:

- (a) Only well-formedness
- (b) Well-formedness **and** conformance to DTD or XML Schema
- (c) No root element
- (d) SOAP Envelope only

Answer: (b)

---

34. The HTTP header **Accept** tells the server:

- (a) The client's password
- (b) Which media types the client can handle in the response
- (c) The database connection string
- (d) The SOAP Fault actor

Answer: (b)

---

35. JAX-RS is used for:

- (a) SOAP web services
- (b) RESTful web services (e.g. Jersey, RESTeasy)
- (c) UDDI only
- (d) CORBA IDL

Answer: (b)

SECTION B — STRUCTURED QUESTIONS (35 marks)

B1. Distributed computing essentials **(5 marks)**

(a) Write one sentence capturing the essence of distributed computing. **(1 mark)**
(b) Briefly explain **Performance** and **Resilience/redundancy** as benefits. **(4 marks)**

Answer

(a) Distributed computing makes multiple networked computers work together as one coordinated system, with software components communicating via messages to achieve a common task.

(b) Performance: Tasks run in parallel across nodes; load is distributed across servers, improving throughput and efficiency. **Resilience/redundancy:** Multiple machines can offer the same service; if one fails or a data centre is down, others can continue, improving availability.

B2. URL and HTTP request line **(5 marks)**

For: `https://api.school.edu:8443/v1/students?name=Ann#profile`

(a) Name the **scheme**, **host**, **port**, **path**, **query**, and **fragment**. **(3 marks)**
(b) State the three parts of an HTTP **request line**. **(2 marks)**

Answer

(a) Scheme: `https` | Host: `api.school.edu` | Port: `8443` | Path: `/v1/students` | Query: `?name=Ann` | Fragment: `#profile`

(b) Method (e.g. GET), **Request-URI** (path + optional query), **HTTP version** (e.g. HTTP/1.1).

B3. SOA roles and publish-find-bind **(5 marks)**

(a) Name the three SOA roles. **(1 mark)**
(b) For each role, state one responsibility. **(3 marks)**
(c) What interaction does the consumer perform **after** finding a service in the registry? **(1 mark)**

Answer

(a) Service Provider, Service Broker/Registry, Service Requester (Consumer).

(b) Provider: develops, deploys, publishes service description (WSDL) to registry. **Registry:** stores and enables discovery of services. **Consumer:** finds service, obtains WSDL, binds and invokes.

(c) Bind and invoke the provider's endpoint (e.g. send SOAP/HTTP requests).

B4. SOAP message structure and abbreviations **(5 marks)**

(a) Expand: **SOAP**, **WSDL**, **UDDI**. **(3 marks)**
(b) State role of **Envelope**, **Header**, **Body**; which are optional vs required? **(2 marks)**

Answer

(a) Simple Object Access Protocol | Web Services Description Language | Universal Description, Discovery, and Integration.

(b) Envelope: root wrapper (required). **Header:** optional metadata (auth, routing). **Body:** payload (required). Optional: **Fault** inside Body on errors.

B5. RPC vs Document SOAP; REST verbs **(5 marks)**

(a) Give **two** differences between RPC-style and Document-style SOAP. **(2 marks)**
(b) List the four REST verbs and matching CRUD operations. **(3 marks)**

Answer

(a) RPC: operation wrapper + parameters like a remote call; Document: full XML document in Body, industry default in JAX-WS. Document supports complex types better.

(b) GET–Read | POST–Create | PUT–Update (replace) | DELETE–Delete.

B6. JAX-WS TimeServer pattern **(5 marks)**

Match each fragment to **SEI**, **SIB**, **Publisher**, or **Client**:

1. Interface with `@WebMethod` and `@SOAPBinding`
2. Class implementing that interface with `@WebService(endpointInterface=...)`
3. `main` with `Endpoint.publish(url, new Impl())`
4. `Service.create(url, qname)` and `getPort(SEI.class)`

Also: what are the **two parameters** of `Endpoint.publish`?

Answer

1. **SEI** | 2. **SIB** | 3. **Publisher** | 4. **Client**

Endpoint.publish: (1) publication **URL**; (2) **SIB instance** that handles requests.

B7. `wsimport` vs `wsgen`; WSDL elements **(5 marks)**

(a) Contrast `wsgen` and `wsimport` (who uses each, input, approach). **(3 marks)**
(b) State the purpose of WSDL elements `<types>` and `<binding>`. **(2 marks)**

Answer

(a) wsgen: provider, bottom-up, input = `@WebService` class, may output WSDL/JAXB artifacts. **wsimport:** consumer, top-down, input = WSDL URL/file, output = client stubs (SEI, Service, JAXB).

(b) types: XSD data types for messages. **binding:** links abstract operations to concrete protocol (e.g. SOAP over HTTP).

SECTION C — ESSAY QUESTIONS (70 marks)

C1. Distributed computing evolution to web services **(20 marks)**

Discuss: (a) what distributed computing is and key benefits (performance, resilience, reuse); (b) CORBA, DCOM, and Java RMI as distributed object technologies; (c) limitations that motivated web services and HTTP/XML; (d) role of MOM as an alternative coupling model.

Model answer (outline for full essay)

Introduction: Distributed systems spread components across networked machines that coordinate via messages while appearing as one logical system.

(a) Benefits: Parallelism improves performance; redundant nodes and sites improve resilience; shared services improve reuse and reduce duplicate development.

(b) Object models: CORBA (OMG, language-neutral), DCOM (Microsoft/Windows), RMI (Java/JVM, JRMP). All use largely **synchronous RPC-style** coupling.

(c) Limitations: Firewall-unfriendly protocols, heterogeneous stub maintenance, poor cross-vendor interoperability (e.g. DCOM client to RMI server), weak fit for Internet-scale B2B. **HTTP + XML** became practical standards.

(d) MOM: Asynchronous, queue-based, loosely coupled — sender does not wait; contrasts with OOD/RPC. Led toward message-based integration alongside eventual **web services (SOAP/REST)**.

Conclusion: Web services realise SOA with standards (SOAP, WSDL, UDDI, REST) better suited to Internet integration than early object middleware alone.

C2. SOAP, WSDL, UDDI, and the publish-find-bind cycle **(25 marks)**

Explain how SOAP, WSDL, and UDDI work together in SOA. Include SOAP message structure, main WSDL elements, UDDI's role, and a step-by-step publish-find-bind workflow. Comment on whether this stack is still typical in modern APIs.

Model answer (outline)

SOA roles: Provider, Registry, Consumer.

WSDL: Contract — `<types>`, `<message>`, `<portType>`, `<binding>`, `<service>`, `<port>` (endpoint URL). Answers what, where, how.

UDDI: Directory for publishing and discovering service descriptions (often WSDL references).

SOAP: XML messaging — **Envelope** (required), **Header** (optional), **Body** (required), **Fault** (errors). Request/response over HTTP (or other transports).

Workflow:

1. Provider implements service, generates/publishes WSDL.
2. Provider **registers** in UDDI.
3. Consumer **queries** UDDI.
4. Consumer retrieves **WSDL**, builds client (`wsimport` or manual).
5. Consumer **invokes** via SOAP messages.

Modern note: UDDI less common publicly; REST + OpenAPI widely used; SOAP/WSDL remain in enterprise, banking, legacy integration.

C3. Critical comparison: SOAP vs REST **(25 marks)**

Compare SOAP and REST across: message format, service description, protocol, state, security, performance, error handling. Recommend one for (i) hospital patient records integration between legacy systems, (ii) a public weather API for mobile apps. Justify.

Model answer (outline)

Dimension	SOAP	REST
Format	XML envelopes	JSON/XML, flexible
Description	WSDL (formal)	OpenAPI optional
Protocol	HTTP, SMTP, JMS…	HTTP/HTTPS
State	Can be stateful	Stateless
Security	WS-Security (message-level)	HTTPS, OAuth, JWT
Performance	Heavier, less cacheable	Lighter, HTTP cache
Errors	`<Fault>` element	HTTP status + body

(i) Hospital legacy integration: SOAP — strict contracts (WSDL), WS-Security, ACID/transaction standards, formal interoperability between heterogeneous legacy systems.

(ii) Public weather API: REST — JSON, caching, scalability, easy mobile consumption, stateless horizontal scaling.

Conclusion: Choice is strategic — security, contract rigidity, and legacy vs speed and public consumption.

C4. JAX-WS SOAP development lifecycle **(20 marks)**

Describe the full lifecycle of a JAX-WS SOAP service: SEI, SIB, annotations (`@WebService`, `@WebMethod`, `@SOAPBinding`), publishing with `Endpoint.publish`, WSDL at `?wsdl`, RPC vs Document styles, and client

consumption with and without `wsimport`. Include roles of `wsgen` and `wsimport`.

Model answer (outline)

1. Design SEI: Interface with `@WebService`, `@WebMethod`, optional `@SOAPBinding(RPC|DOCUMENT)`.

2. Implement SIB: Class implements SEI; `@WebService(endpointInterface="...")`; business logic in methods.

3. Publish: `Endpoint.publish("http://host:port/path", new SIB())` — starts HTTP listener; WSDL at `?wsdl`.

4. RPC vs Document: RPC maps parameters in Body; Document sends XML documents; **Document is default**.

5. Client without wsimport: `URL(?wsdl)`, `QName(ns, serviceName)`, `Service.create`, `getPort(SEI.class)`, call methods.

6. Client with wsimport: Top-down from WSDL; generated `Service` class and port; cleaner for consumers without source.

wsgen: Provider, bottom-up, optional physical WSDL. **wsimport:** Consumer, top-down stubs.

Reflection: Separation of contract (SEI/WSDL) and implementation (SIB) supports loose coupling and exam-style TimeServer pattern.

MARKING GUIDE (INSTRUCTOR)

Section	Marks	Guidance
A — MCQ	35	1 mark each; no partial credit
B1–B7	35	~5 marks each; partial credit for correct fragments
C1	20	Benefits/tech/MOM coverage
C2	25	SOA + three standards + workflow
C3	25	Comparison table + two scenarios
C4	20	JAX-WS lifecycle + tools

Suggested pass: 70/140 (50%) — adjust per department policy.

QUICK REVISION CHECKLIST

- [ ] URL parts: scheme, host, port, path, query, fragment
- [ ] HTTP request line; common headers (Host, Content-Type, Accept)
- [ ] Distributed benefits; CORBA / DCOM / RMI / MOM

- [ ] Web service vs API; SOA three roles; UDDI publish-find-bind
- [ ] SOAP Envelope/Header/Body/Fault; RPC vs Document
- [ ] WSDL elements; wsgen vs wsimport; SEI vs SIB
- [ ] REST verbs + CRUD; SOAP vs REST comparison
- [ ] `Endpoint.publish`, `QName`, `Service.create`, `getPort`
- [ ] Simple HTTP client: host, path, request string, Socket

End of IST604 Practice Examination (140 marks).